

IST604 – Web Services & Distributed Computing

Chapter 3: Developing Web Services Using SOAP — Structural & Essay Practice Questions

Master's-level examination preparation. Structural questions test precise, concise knowledge; essay questions require critical analysis, design justification, and code-level reasoning.

Part I: Structural / Short-Answer Questions

Group 1: SOAP Transmission and Standards

Q1. Describe the three roles involved in SOAP message transmission and explain the function of each. Include a diagram showing the message path. (6 marks)

Model Answer Guidance:

- **SOAP Sender:** creates and initiates the SOAP message toward the ultimate receiver.
- **SOAP Intermediary:** optional node(s) along the delivery path; can intercept and process messages (filtering, logging, routing, caching). Multiple intermediaries can exist in a chain.
- **Ultimate SOAP Receiver:** the final intended destination that fully processes the message.
- Diagram: `SOAP Sender → Intermediary → Intermediary → Ultimate SOAP Receiver`

Q2. Explain the role of WSDL in a SOAP-based web services architecture. What three questions does a WSDL document answer about a web service? (4 marks)

Model Answer Guidance: WSDL is the metadata/contract language. It answers: (1) **What** the service does (operations/methods), (2) **Where** the service is located (endpoint URL), (3) **How** to access it (protocol, data format, binding). It is machine-readable and used by tools like `wsimport` to generate stubs automatically.

Q3. Distinguish between SEI and SIB in JAX-WS development, including the Java annotations used in each. (6 marks)

Model Answer Guidance:

- **SEI** (Service Endpoint Interface): a Java interface annotated with `@WebService` and `@SOAPBinding`; each operation method annotated with `@WebMethod`. Declares *what* operations are available.
- **SIB** (Service Implementation Bean): a Java class annotated with `@WebService(endpointInterface="pkg.InterfaceName")`; implements the SEI methods. Contains the actual business logic. Can be a POJO or Stateless Session EJB.

Q4. Write the Java code for publishing a web service named `CalculatorImplementation` at `http://localhost:8081/ws/calculator`. Identify which class and method are used and explain each

argument. (4 marks)

Model Answer Guidance:

```
import javax.xml.ws.Endpoint;
public class CalculatorWS {
    public static void main(String[] args) {
        Endpoint.publish("http://localhost:8081/ws/calculator",
            new CalculatorImplementation());
    }
}
```

- `Endpoint.publish()` is the JAX-WS method for publishing.
- First argument: the publication URL string.
- Second argument: an instance of the SIB.

Q5. A developer runs a JAX-WS service on JDK 17 and receives a `WebServiceProvider` annotation error. Explain the cause and the solution. (4 marks)

Model Answer Guidance: From Java 9 onwards, certain APIs (including `javax.annotation`) were removed from the default JDK bundling due to modularisation (Project Jigsaw). The `javax.annotation-api` JAR is no longer included by default. **Solution:** download the `javax.annotation-api` JAR from the Maven Repository (e.g., version 1.3.2) and add it manually to the project's classpath.

Group 2: Client Programming

Q6. Describe the five-step pattern used in a JAX-WS Java client to invoke a remote SOAP web service without using `wsimport`. Include the key classes and methods involved. (10 marks)

Model Answer Guidance:

1. Create a `URL` object pointing to the service's WSDL: `new URL("http://host:port/path?wsdl")`
2. Create a `QName` identifying the service: `new QName("namespace", "ServiceName")`
3. Create the `Service` factory: `Service.create(url, qname)`
4. Extract the port (proxy object): `service.getPort(XxxInterface.class)`
5. Call methods on the port as if local: `port.someMethod(args)`

Q7. Compare the two client approaches demonstrated in the Order Processing Service example: the direct `QName`-based client (OrderClient1) and the `wsimport`-generated stub client (OrderClient2). What are the advantages and limitations of each? (6 marks)

Model Answer Guidance:

- **OrderClient1** (Direct): requires manual `QName`, namespace string, and WSDL URL construction. More verbose; developer must read the WSDL to extract service name and namespace.

Advantage: no pre-generation step needed.

- **OrderClient2** (wsimport stubs): uses auto-generated `OrderServiceImplService` class and typed port. Three-line invocation pattern. Cleaner, safer (compile-time type checking), and no manual WSDL inspection needed. Limitation: requires running `wsimport` first; generated stubs must be regenerated if the WSDL changes.

Group 3: wsgen and wsimport

Q8. Explain the difference between `wsgen` and `wsimport`, specifying the input, output, approach, and typical user of each tool. (8 marks)

Model Answer Guidance:

	wsgen	wsimport
Approach	Bottom-Up (Code-First)	Top-Down (Contract-First)
Input	Compiled <code>.class</code> file with <code>@WebService</code>	<code>.wsdl</code> file (local or URL)
Output	JAXB classes; optionally WSDL with <code>-wsdl</code> flag	SEI, Service class, JAXB data beans
User	Service Provider	Service Consumer/Client

Note: Since Java 6, the JAX-WS runtime can dynamically generate `wsimport` artifacts at publish time for simple services; `wsimport` is mainly needed when a physical WSDL file must be shared pre-deployment.

Q9. A client developer receives the following WSDL URL:

`http://api.company.com/ws/inventory?wsdl`

They want to generate Java stubs in the package `com.company.client` and keep the source files. Write the complete `wsimport` command. (4 marks)

Model Answer Guidance:

```
wsimport -keep -s ./src -p com.company.client
http://api.company.com/ws/inventory?wsdl
```

Explanation:

- `-keep`: retains generated `.java` source files
- `-s ./src`: places generated sources in the `src` directory
- `-p com.company.client`: sets the Java package for generated classes
- Final argument: the WSDL URL

Group 4: Document Style vs RPC Style

Q10. Compare RPC style and Document style SOAP binding in JAX-WS across four dimensions: message structure, supported data types, industry status, and Java annotation. (8 marks)

Model Answer Guidance:

Dimension	RPC Style	Document Style
Message structure	Mimics function call; parameters mapped individually into the SOAP body	Sends a complete XML document as the SOAP body
Supported data types	Simple types only (String, int, long, etc.)	Simple and complex/rich types; full objects
Industry status	Simpler but less flexible	Industry standard ; default in JAX-WS
Java annotation	<code>@SOAPBinding(style = Style.RPC)</code>	<code>@SOAPBinding(style = Style.DOCUMENT)</code> or omit

Q11. Write the complete Service Endpoint Interface (SEI) for a `TemperatureConverter` web service using **Document style** that exposes one operation: `double celsiusToFahrenheit(double celsius)`. (5 marks)

Model Answer Guidance:

```
package JWS;
import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.jws.soap.SOAPBinding;

@WebService
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public interface TemperatureConverterInterface {
    @WebMethod
    double celsiusToFahrenheit(double celsius);
}
```

Q12. For the Calculator web service, the WSDL is accessed at `http://localhost:8081/ws/calculator?wsdl`. From the WSDL, a developer identifies: service name = `CalculatorImplementationService`, namespace = `http://JWS/`. Write the Java client code to invoke the `add(10, 25)` operation using the direct (non-wsimport) approach. (6 marks)

Model Answer Guidance:

```
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;

public class CalculatorClient {
    public static void main(String[] args) throws Exception {
        URL url = new URL("http://localhost:8081/ws/calculator?wsdl");
```

```

        QName qname = new QName("http://JWS/",
"CalculatorImplementationService");
        Service service = Service.create(url, qname);
        CalculatorInterface calc = service.getPort(CalculatorInterface.class);
        System.out.println("Result: " + calc.add(10, 25));
    }
}

```

Part II: Essay Questions

Essay 1 — Architecture and Implementation

Q13. "A well-structured JAX-WS web service is more than just an annotated Java class."

Discuss the complete architecture of a JAX-WS SOAP-based web service implementation. In your answer, address: (a) the roles and responsibilities of the SEI and SIB; (b) the Java annotations required and their significance; (c) the publishing mechanism; (d) the role of the auto-generated WSDL; and (e) how a Java client consumes the service, both with and without `wsimport`. (25 marks)

Essay Guidance:

- **Intro:** Frame the separation of interface (SEI) from implementation (SIB) as a reflection of good object-oriented design and the loose-coupling principle.
- **SEI:** `@WebService`, `@SOAPBinding`, `@WebMethod` — declares the service contract; analogous to the WSDL `<portType>`. POJO vs EJB trade-offs.
- **SIB:** `@WebService(endpointInterface=...)` — links to the SEI; contains business logic; can run in Java SE (lightweight) or Application Server (EJB for production scalability).
- **Annotations deep dive:** `@SOAPBinding` determines message format; `@WebMethod` controls operation exposure; `endpointInterface` ensures tight binding to the contract.
- **Publishing:** `Endpoint.publish(url, sib)` — Java SE's built-in HTTP server sufficient for development. Production: GlassFish, Tomcat, JBoss, WebSphere.
- **Auto-generated WSDL:** accessing `?wsdl` shows the machine-generated contract; contains `<types>`, `<message>`, `<portType>`, `<binding>`, `<service>`.
- **Client without wsimport:** URL → QName → Service.create() → getPort() → method call. Verbose; requires reading the WSDL manually.
- **Client with wsimport:** single command generates SEI + Service class; 3-step pattern (new Service(), getPort(), call). Cleaner, type-safe, preferred in practice.
- **Conclusion:** The 4-file pattern (SEI, SIB, Publisher, Client) mirrors the SOA roles of provider and consumer, and the WSDL bridges the gap.

Essay 2 — Comparative Analysis: RPC Style vs Document Style

Q14. Analyse the differences between RPC style and Document style SOAP binding in JAX-WS. Your answer should cover: the conceptual difference in how messages are constructed; the constraints each imposes on data types; their respective industry adoption; the annotation differences; and the practical implications for a developer choosing between them. Illustrate with reference to the examples from the lecture. (20 marks)

Essay Guidance:

- **Conceptual:** RPC style maps Java method calls directly to SOAP body elements — each parameter becomes a child element. Document style wraps everything in a single XML document — more flexible but more verbose.
- **Data type constraints:** RPC limited to primitives/simple types (String, int, long). Document style supports rich Java objects mapped to XSD complex types via JAXB — enables sending **Order** objects, lists, nested structures.
- **Industry adoption:** Document style is the W3C/industry default and the WS-I Basic Profile recommendation. RPC is simpler for learning but discouraged in production due to interoperability issues.
- **Annotations:** **Style.RPC** vs **Style.DOCUMENT** (or omitting **@SOAPBinding** entirely defaults to DOCUMENT).
- **From the lecture:** TimeServer, HelloWorld, Calculator all use RPC (simple return types). OrderProcessing uses DOCUMENT — more realistic enterprise scenario.
- **Developer implications:** DOCUMENT requires more WSDL/JAXB configuration; generates more verbose XML; but is necessary for enterprise-grade services that exchange complex business objects.
- **Conclusion:** For real-world applications involving complex data, Document style is the correct choice; RPC style is useful only for pedagogical simplicity or very trivial services.

Essay 3 — wsgen, wsimport, and the Development Lifecycle

Q15. The tools **wsgen** and **wsimport** represent two opposite ends of the SOAP web service development lifecycle. Discuss the role each tool plays, when each should be used, and how they relate to the Bottom-Up and Top-Down development approaches. Use concrete examples from the lecture to support your answer, and reflect on the practical implications for team-based enterprise development where service providers and consumers may be in different organisations. (20 marks)

Essay Guidance:

- **wsgen (Bottom-Up):** input is compiled SIB; output is JAXB bindings and optionally a WSDL. Used when service logic already exists. Generates deployment artifacts. Note: JAX-WS runtime can do this automatically at publish time for simple cases, making explicit **wsgen** calls mostly needed for pre-deployment WSDL sharing.
- **wsimport (Top-Down):** input is a WSDL URL or file; output is SEI, Service class, JAXB beans. Used by client developers who have no access to service source code.
- **Lifecycle connection:** Provider runs **wsgen** (or publishes and shares WSDL URL) → Client developer runs **wsimport** against that WSDL → Generated stubs allow calling the service as if it were local.
- **From lecture:** CalcClient.java and OrderClient2 demonstrate **wsimport** usage; the Calculator and Order services are the provider side.
- **Enterprise/team implications:** In cross-organisational integration, the WSDL is the **only shared contract**. The consuming team has no source access — **wsimport** is the only path. This reinforces why the WSDL must be carefully designed and versioned. Changes to WSDL break all consumers' generated stubs, making WSDL versioning a critical governance concern.

- **Conclusion:** These tools operationalise the publish-find-bind SOA cycle: `wsgen` enables publishing, `wsimport` enables binding. Together, they make JAX-WS interoperable across language and organisational boundaries.

Essay 4 — Critical Design Question

Q16. You are tasked with building a SOAP-based web service for a hospital system that exposes patient record operations: `getPatientRecord(int patientId)` which returns a complex `Patient` object containing name, date of birth, diagnosis list, and medication list.

- Explain why Document style is more appropriate than RPC style for this service. (5 marks)
- Design the complete SEI for this service, including all necessary JAX-WS annotations. (5 marks)
- Describe the four-file structure of the full service implementation, specifying the role of each file. (4 marks)
- A third-party Java application needs to consume this service. Describe the process from WSDL generation to client invocation using `wsimport`. (6 marks)

Essay Guidance:

(a) RPC style only supports simple types; `Patient` is a complex object with nested lists — JAXB-mapped to XSD complex types requires Document style. Document style is also the WS-I profile standard for enterprise services, essential in healthcare interoperability (e.g., HL7 FHIR-compliant gateways). Security and compliance requirements in healthcare further favour Document style's richer message-level processing support.

(b) SEI design:

```
@WebService
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)
public interface PatientServiceInterface {
    @WebMethod
    Patient getPatientRecord(int patientId);
}
// Patient would be a JAXB-annotated class with fields:
// String name, Date dob, List<String> diagnoses, List<String> medications
```

(c) Four files:

- `PatientServiceInterface.java` (SEI): declares `@WebService`, `@WebMethod`, `@SOAPBinding(DOCUMENT)`
- `PatientServiceImpl.java` (SIB): `@WebService(endpointInterface=...)`, implements `getPatientRecord()`
- `PatientServicePublisher.java`: calls `Endpoint.publish(url, new PatientServiceImpl())`
- `PatientClient.java`: connects using either direct `Service.create()` or `wsimport` stubs

(d) `wsimport` process:

1. Provider publishes service; WSDL accessible at `http://host:port/path?wsdl`
2. Client developer runs: `wsimport -keep -s ./src -p com.hospital.client http://host:port/ws/patients?wsdl`
3. Generated classes: `PatientServiceImplService.java`, `PatientServiceInterface.java`, `Patient.java` (JAXB bean)
4. Client invocation:

```
PatientServiceImplService service = new PatientServiceImplService();
PatientServiceInterface port = service.getPatientServiceImplPort();
Patient p = port.getPatientRecord(12345);
```

End of Chapter 3 Practice Questions